

# LC 675: Cut Off Trees for Golf Event

Companies: Apple, Flipkart

## Grid Rules

**0** → cant be walked

**1** → can be walked

**>1** → a tree needs to be cut

**Note:** trees must be cut from shortest to tallest, once cut it values becomes 1, start = {0,0}

## Algorithm Steps

**(S1)** store all trees → height, row, col and then sort by height

**(S2)** for every consecutive tree → run BFS

current pos → BFS → next tree  
and then **current pos = tree pos**



Since no weighted edges, it's a normal BFS and will need only a queue and a seen

# C++ Implementation

---

```

class Solution {
public:
    int n;
    int m;
    vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

    int bfs(int sr, int sc, int er, int ec, vector<vector<int>>& forest){
        queue<pair<int,int>> q;
        vector<vector<int>> seen(n, vector<int>(m, 0));

        q.push({sr,sc});
        seen[sr][sc]=1;
        int dist=0;

        while(!q.empty()){
            int currsz=q.size();
            while(currsz--){
                int r=q.front().first;
                int c=q.front().second;

                q.pop();

                if(r==er && c==ec) return dist;

                for(auto &it:dir){
                    int nr=r+it.first;
                    int nc=c+it.second;
                    if(nr>=0 && nc>=0 && nr<n && nc<m && !seen[nr][nc] && forest[nr][nc]
                    seen[nr][nc]=1;
                    q.push({nr,nc});
                }
            }
            dist++;
        }
        return -1;
    }

    int cutOffTree(vector<vector<int>>& forest) {
        n=forest.size();
        m=forest[0].size();

        //lets store trees so that unko like heightwise sort kar paaye
        //phir 0,0 se ek ek kar ke bfs krenge tree to tree

        //{height,{tr,tc}}
        vector<pair<int,pair<int,int>>> trees;

```

```
for(int i=0;i<n;i++){
    for(int j=0;j<m;j++){
        if(forest[i][j]>1){
            trees.push_back({forest[i][j],{i,j}});
        }
    }
}

sort(trees.begin(),trees.end());
int ans=0;
int currr=0;
int currc=0;

for(auto &it:trees){
    int tr=it.second.first;
    int tc=it.second.second;

    int dist=bfs(currr,currc,tr,tc,forest);
    if(dist!=-1){
        ans+=dist;
    }
    else{
        return -1;
    }
    currr=tr;
    currc=tc;
}

return ans;
}

};
```